Outline

What is feature engineering?

Three related-but-distinct activities on tabular data:

- **Preprocessing (L01c)** - *Fix* the data. Same columns, cleaner.

```
df["rent"] = df["rent"].fillna(df["rent"].median())
```

- **Feature engineering** - *Create* new columns. More columns.

```
df["price_per_m2"] = df["rent"] / df["area"]
```

- **Feature selection (L01h)** - *Choose* which to keep. Fewer columns.

```
X = X[["price_per_m2", "rooms", "district_te"]]
```

The classical-ML maxim: "Good features beat a better algorithm." Most Kaggle winners come from FE, not the model class. *Deep learning* learns features from raw inputs (pixels, tokens, audio); for tabular data, hand-engineered features still dominate.

Predict-first: where does an ML project's time go?

An ML project has several stages: data collection, EDA, preprocessing, feature engineering, modeling, tuning, evaluation, deployment.

Predict-first: where does an ML project's time go?

An ML project has several stages: data collection, EDA, preprocessing, feature engineering, modeling, tuning, evaluation, deployment.

Predict: what fraction of project time is spent on data & features vs modeling & tuning?

Predict-first: where does an ML project's time go?

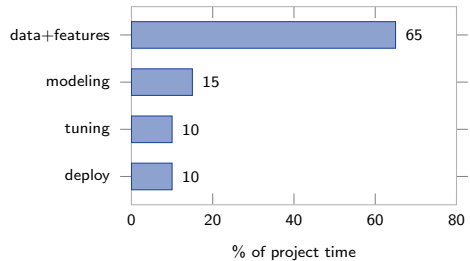
An ML project has several stages: data collection, EDA, preprocessing, feature engineering, modeling, tuning, evaluation, deployment.

Predict: what fraction of project time is spent on data & features vs modeling & tuning?

Surveys (CrowdFlower 2016, Anaconda 2020) consistently report:

- **60-80%** on data collection, cleaning, feature engineering.
- **10-20%** on modeling and tuning.
- **5-15%** on deployment and monitoring.

Students often expect the reverse. The reality: *the algorithm is the easy part*; the data preparation is the work.



Sources of new features

Three super-categories. Yerevan rent examples in italics.

From the same row
(one observation in, one feature out)

- Transformations: log, power, binning. *log(rent)*
- Combinations: ratios, differences, products. *rent / area, year_listed − year_built*
- Polynomial / interactions. *area × floor*

From many rows
(group of observations → feature)

- Group-by aggregations. *avg_rent_by_district*
- Rolling windows (trailing). *mean rent over last 30 days*
- Lags. *rent at the previous listing*
- Density / k-NN. *listings within 1 km*

From outside the data
(joins to external sources)

- Geospatial joins. *distance to Republic Square*
- Temporal joins. *is_holiday(listed_at)*
- Demographic joins. *district median income*
- Domain reference data. *m^2 prices from official index*

Domain knowledge cuts across all three - the biggest single multiplier. The rest of this deck walks through each category.

Numerical transformations

Log / power. For skewed positive targets (recap L01c). `np.log1p(price)` compresses the tail.

Binning. Convert a continuous feature into categories. Three strategies:

- Equal-width: same range per bin.
- Equal-frequency (quantile): same count per bin.
- Supervised: pick splits that maximize separation in y.

When to bin? When the target depends *non-monotonically* on the feature (e.g., rent peaks at 60 m^2 then plateaus) and you're using a linear model. Trees don't need binning.

```python
from sklearn.preprocessing \
  import KBinsDiscretizer

# 5 quantile bins on area
binner = KBinsDiscretizer(
    n_bins=5,
    encode="ordinal",
    strategy="quantile")
df["area_bin"] = binner.
    fit_transform(
    df[["area"]])
```

Polynomial and interaction features

Linear regression on *engineered* features is the most underrated trick in classical ML. From L01b: polynomial regression is OLS with $\phi(x) = (1, x, x^2, \ldots, x^d)$.

Interactions extend this to multiple features. Given x_1, x_2:

$$\phi(\mathbf{x}) = (1,\ x_1,\ x_2,\ x_1^2,\ x_2^2,\ x_1 x_2)$$

The product $x_1 x_2$ lets a linear model express "effect of x_1 depends on level of x_2."

Predict-first: you expect rent to depend on `area` $\times$ `floor`. Do you need to add the product explicitly for: (a) Ridge regression, (b) Random Forest?

Polynomial and interaction features

Linear regression on *engineered* features is the most underrated trick in classical ML. From L01b: polynomial regression is OLS with $\phi(x) = (1, x, x^2, \ldots, x^d)$.

Interactions extend this to multiple features. Given x_1, x_2:

$$\phi(\mathbf{x}) = (1,\ x_1,\ x_2,\ x_1^2,\ x_2^2,\ x_1 x_2)$$

The product $x_1 x_2$ lets a linear model express "effect of x_1 depends on level of x_2."

Predict-first: you expect rent to depend on `area` $\times$ `floor`. Do you need to add the product explicitly for: (a) Ridge regression, (b) Random Forest?

Answers: (a) **Yes** - a linear model can't construct products. (b) **No** - trees discover interactions via nested splits. Polynomial features are a *linear-model crutch*, not a universal good.

Ratios and differences

Many of the most predictive features are simple ratios or differences. Linear models can't construct them; trees can but slowly.

```
# Yerevan-rent examples
df["price_per_m2"]     = df["rent"] /
    df["area"]
df["bedroom_density"] = df["rooms"] /
     df["area"]
df["age_at_listing"]   = (df["
    year_listed"]
                          - df["
                              year_built
                              "])

# Classical examples
df["bmi"]    = df["weight"] / df["
    height"]**2
df["lti"]    = df["loan"] / df["income
    "]
df["debt_to_equity"] = df["debt"] /
    df["equity"]
```

Why ratios beat raw features:

- Often more predictive than either component alone.
- Express the underlying *rate* or *intensity*.
- Scale-invariant - a 3000 USD loan to a 6000 USD earner is the same as 30k/60k.
- Sparse - one engineered feature vs. two raw ones.

When NOT to engineer:

- If the denominator can be 0 or negative.
- For deep learning on tabular - just feed in the raw inputs.

Datetime features

Datetimes are rich. Extract components, time-since-events, calendar markers, and cyclic encodings.

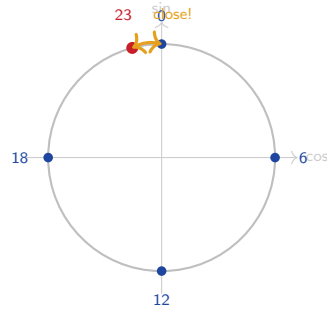
```
df["dt"] = pd.to_datetime(df["listed_at"])
df["year"]    = df["dt"].dt.year
df["month"]   = df["dt"].dt.month
df["weekday"] = df["dt"].dt.dayofweek     # 0=
    Mon
df["is_weekend"]    = df["weekday"].isin([5,
    6])
df["days_since_listed"] = (today - df["dt"]).
    dt.days
df["is_holiday"] = df["dt"].dt.date.isin(
    holidays)

# cyclic encoding for periodic features
df["hour_sin"] = np.sin(2*np.pi * df["hour"
    ]/24)
df["hour_cos"] = np.cos(2*np.pi * df["hour"
    ]/24)
```

Watch **leakage** on "time since last event" - use events strictly before the row's timestamp.

Why cyclic encoding?

Hour 23 and hour 0 are 1 hour apart, not 23. Map onto the unit circle:



Hour 23 (red) lands right next to hour 0 (blue) on the circle.

Group-by aggregations

For each row, attach summary statistics of its group. Hugely effective for tabular ML.

```
# Yerevan: for each apartment, learn the district's price profile
df["avg_rent_district"]    = df.groupby("district")["rent"].transform("mean")
df["med_rent_district"]    = df.groupby("district")["rent"].transform("median
    ")
df["std_rent_district"]    = df.groupby("district")["rent"].transform("std")
df["count_in_district"]    = df.groupby("district")["rent"].transform("count"
    )
df["rank_in_district"]     = df.groupby("district")["rent"].rank(pct=True)
df["rent_vs_district_avg"] = df["rent"] / df["avg_rent_district"]
```

Leakage warning: computing group means on the *full* dataset before splitting leaks the target into features. *Typical inflation in our Yerevan example:* honest CV MAE = 32 kAMD; leaky CV MAE = 24 kAMD - the model looks 25% better than it actually is. Fit aggregations on the train fold only, or use a CV-aware encoder from `category_encoders`.

Time-series features: lags and rolling windows

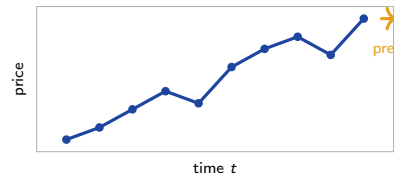
```python
df = df.sort_values("dt")

# lag features
df["lag_1"]  = df["price"].shift(1)
df["lag_7"]  = df["price"].shift(7)

# rolling windows (trailing!)
df["ma_7"]   = df["price"].rolling(7).mean
    ()
df["std_7"] = df["price"].rolling(7).std
    ()

# expanding / EMA
df["ema_7"] = df["price"].ewm(span=7).
    mean()

# diff features
df["diff_1"]   = df["price"].diff(1)
df["pct_chg"]  = df["price"].pct_change()
```



Look-ahead bias: `rolling(7, center=True)` silently uses future values. *Never* center rolling windows when forecasting.

Geospatial features

When you have lat/lon, the raw coordinates are usually useless; the *relationships* are predictive.

```
from math import radians, sin, cos, asin,
     sqrt

def haversine(lat1, lon1, lat2, lon2):
    R = 6371       # km
    dlat = radians(lat2 - lat1)
    dlon = radians(lon2 - lon1)
    a = (sin(dlat/2)**2
         + cos(radians(lat1))
         * cos(radians(lat2))
         * sin(dlon/2)**2)
    return 2 * R * asin(sqrt(a))

# Distance to Republic Square (Yerevan
    center)
df["dist_to_center"] = df.apply(lambda r:
    haversine(r.lat, r.lon, 40.177,
        44.512),
    axis=1)
```

Useful geospatial features:

- **Distances** to landmarks (metro, river, parks).
- **Cluster IDs** from k-means or DBSCAN on (lat, lon).
- **Grid cells** via geohash, H3, or S2 - aggregations within a cell.
- **Density features** (k-NN density estimate around each point).
- **POI counts** within a radius (cafes, schools, public transport).

Raw lat/lon as columns in a linear model: don't. Trees can split on lat/lon but distances/clusters are usually more efficient.

Text features (intro)

Text needs to become numbers. Three layers, increasing in power:

- **Engineered scalars.** Length, word count, punctuation count, ALL-CAPS ratio, emoji count. Cheap and surprisingly predictive for spam, sentiment.
- **Bag-of-words / TF-IDF / n-grams.** Tokenize, count, weight by inverse document frequency. The classical NLP workhorse.
- **Embeddings.** Word2Vec, sentence-BERT, modern transformer encoders. Deferred to the NLP chapter.

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=2000, ngram_range=(1, 2),
                        min_df=5, stop_words="english")
X_text = tfidf.fit_transform(df["description"])

df["desc_length"]  = df["description"].str.len()
df["caps_ratio"]   = df["description"].str.count(r"[A-Z]") / df["desc_length
    "]
df["num_exclaim"]  = df["description"].str.count(r"!")
```

Feature crosses for high-cardinality

When two high-cardinality categoricals matter *jointly*, you can either combine them into one feature (a "cross") or learn an embedding.

```
# Manual cross: district x rooms -> Kentron_3br
df["district_x_rooms"] = df["district"] + "_" + df["rooms"].astype(str)

# Then encode the cross with target / frequency / OHE
```

- Used heavily in classical CTR prediction (Google's Wide & Deep paper, 2016).
- Risk: explosion. 100 districts $\times$ 10 room counts $=$ 1000 categories.
- Modern alternative: learn an *embedding* per categorical, sum or concatenate. NN does this naturally; `category_encoders` has `HashingEncoder` for the classical version.

After engineering: select (L01h preview)

More features is not always better. *Curse of dimensionality* (math 30), multicollinearity (L01b), variance growth, compute cost, and interpretability all push for fewer features.

Three families (full treatment in L01h):

- **Filter** - score each feature independently (`f_regression`, mutual info). Fast but misses interactions.
- **Embedded** - the model selects while fitting. *Lasso*, tree importances, permutation importance.
- **Wrapper** - search subsets via CV. *RFE-CV*, sequential forward/backward, Boruta. Slow but accurate.

```
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.linear_model import LassoCV

# Filter: top 20 by linear correlation with y
SelectKBest(f_regression, k=20).fit(X, y)
# Embedded: L1 forces sparsity
lasso = LassoCV(cv=5).fit(X, y)          # zero coefficients = dropped
```

Workflow: filter for initial cut ($p > 10{,}000 \rightarrow 500$), embedded for production ($500 \rightarrow 50$), wrapper only when stakes justify the compute. L01h covers each family in depth.

Automated feature engineering

Tools that generate features automatically from your raw data:

- **featuretools.** "Deep Feature Synthesis" walks relational schemas and generates aggregations, transformations, and combinations.
- **tsfresh.** Extracts hundreds of statistical features from time-series.
- **autofeat, Featurewiz.** General-purpose automated FE.

```
import featuretools as ft
es = ft.EntitySet(id="rent")
es = es.add_dataframe(dataframe=df, dataframe_name="listings", index="id")
es = es.add_dataframe(dataframe=districts, dataframe_name="districts", ...)
feature_matrix, defs = ft.dfs(entityset=es,
                              target_dataframe_name="listings",
                              max_depth=2)
```

What DFS produces (concrete). Given table `listings` with foreign key to `landlords`, DFS generates: `MEAN(landlords.rating)`, `COUNT(landlords.listings)`, `STD(landlords.rent)`, `MAX(landlords.years_active)`, plus transformations like `DAY(dt)`, `MONTH(dt)`. Hundreds of candidates from a small schema.

When to use: early exploration, complex relational data, generating candidates.

When NOT to use: production (hard to maintain, slow at inference). Auto-generated features usually *lose* to a handful of domain features.

Six feature-engineering pitfalls

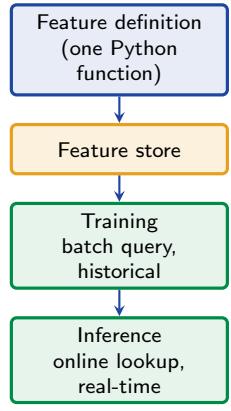
1. **Target leakage.** Computing a target-derived feature (mean rent by district) on the *full* dataset before splitting. Use train fold only or a CV-aware encoder.
2. **Look-ahead bias.** For time-series, using future values in a "rolling" or aggregate feature. Always trailing windows; never `center=True`.
3. **Train-serve skew.** A feature easy to compute on training data but impossible at inference (e.g., "count of total purchases" when the user's full history isn't known yet at inference time).
4. **Hardcoded statistics.** "Just divide by the mean rent" - whose mean? Save the fitted statistic via a Pipeline (L01c); don't recompute on test.
5. **Over-engineering.** 500 features on 1000 rows. Curse of dimensionality + multicollinearity. Stop when CV stops improving; verify via L01h.
6. **Engineering features the model already discovers.** Adding $x_1 x_2$ to a Random Forest is wasted compute - the tree finds that interaction via nested splits. *Skip polynomial features for tree models.* Always check feature importance (L01h SHAP / permutation) before adding more.

Feature stores: training and inference see the same features

In production the model you train and the model you serve see different code paths. Without care, they compute features differently.

A feature store:

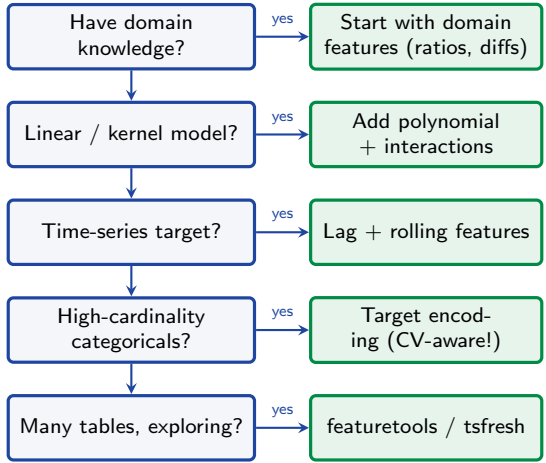
- Stores feature definitions (one function, one source of truth).
- Serves **batch** queries for training (historical, point-in-time).
- Serves **online** queries for inference (low-latency lookup).
- Guarantees train/serve consistency.
- Versions features over time.
- Enforces point-in-time correctness for time-series (no leakage).



Tools: **Feast** (open source), **Tecton**, **Vertex AI FS**, **SageMaker FS**.

For coursework: a Pipeline (L01c) gets you most of the way. For deployed models with feedback loops: you need a real feature store.

Decision flowchart



After engineering, always do a feature-selection pass (Lasso or tree importance) and check CV improvement before keeping.

Recap

- **60-80% of an ML project** is data and features, not modeling. "Good features beat a better algorithm" on tabular data.
- **Domain knowledge is the multiplier.** Ratios (BMI, price-per-m^2), differences (age, days-since), and rates beat raw features.
- **Transformations:** log/power for skew, binning for non-monotonic effects, polynomial + interactions for linear models (*not for trees*).
- **Aggregations:** group-by stats, rolling windows (trailing only), expanding stats. The leakage hides in the fit-on-full-data step.
- **Specialty domains:** datetime (cyclic, components), geospatial (distances, clusters), text (TF-IDF), high-cardinality crosses.
- **Bias-variance lens (L01d2):** each new feature lowers bias and raises variance. Stop when CV stops improving.
- **Selection (L01h):** filter $\rightarrow$ embedded $\rightarrow$ wrapper, in order of cost. Always do a feature-selection pass after engineering.
- **Automation tools** (featuretools, tsfresh) for exploration, not production. Domain features still win.
- **Pitfalls:** target leakage, look-ahead bias, train-serve skew, engineering what the model already discovers. Wrap everything in a Pipeline (L01c); serve from a feature store in production.

HW01g - practice

Estimated time: 90-120 minutes. Tasks 1-4 graded; bonuses optional.

1. On the Yerevan rent dataset, engineer 4-5 domain features by hand (`price_per_m2`, `age_at_listing`, `avg_rent_district`, `rent_vs_district_avg`). *If lat/lon is available, add `dist_to_center`.* Fit Ridge with and without; report test MAE.
2. Add `PolynomialFeatures(degree=2)` to the original feature set. Compare CV score to step 1.
3. Fit Lasso on the union of original + engineered + polynomial features. Which features survive? Compare to manual-only.
4. Replace Ridge with a tree-based model (`HistGradientBoostingRegressor`). Drop polynomial features (recall pitfall #6). Compare to linear pipeline.
5. *Bonus 1:* compute `avg_rent_district` the wrong way (full data, then split). Verify the $\sim 25\%$ inflation from frame 9.
6. *Bonus 2:* use `featuretools`. Compare auto-features to manual. Which wins on test? **Most instructive.**
7. *Bonus 3:* rolling 30-day average rent feature with listings sorted by date. Verify no look-ahead bias.
8. *Bonus 4:* use SHAP / permutation importance (L01h). Drop near-zero-importance features. Does CV improve?